RAMSES Data Structure and the LaRT Interface

Contents

1. Overview of RAMSES

RAMSES [1] is an adaptive mesh refinement (AMR) code for astrophysical hydrodynamics and N-body dynamics. It decomposes the computational domain into an octree hierarchy of grids, refining regions of interest to arbitrarily high resolution. The MPI decomposition uses a space-filling curve (Peano–Hilbert ordering) to distribute octs among CPU ranks.

`LaRT v2.00` can read RAMSES binary output directly via `amr_type = 'ramses'`. This document describes the RAMSES binary format in detail and explains how `read_ramses_amr.f90` translates it into the flat leaf-cell arrays required by the LaRT octree engine.

2. Output Directory Layout

A RAMSES snapshot is stored in a directory named `output_NNNNN/` where `NNNNN` is the five-digit output number. For a run with N_{cpu} MPI tasks the snapshot contains the following per-CPU files:

```
output_00010/
    info_00010.txt           ! global metadata: units, cosmology, etc.
    amr_00010.out00001       ! AMR tree for CPU 1
    amr_00010.out00002       !   ...
    ...
    amr_00010.outNNNNN       ! AMR tree for CPU N
    hydro_00010.out00001     ! hydrodynamic variables for CPU 1
    hydro_00010.out00002
    ...
    hydro_00010.outNNNNN
    gravity_00010.out00001   ! gravitational potential (optional)
    part_00010.out00001      ! particle data (optional)
```

All `.out*` files are Fortran *unformatted sequential* binary files (each record bracketed by 4-byte integer record markers on little-endian systems).

3. The Info File (`info_NNNNN.txt`)

`info_NNNNN.txt` is a plain-text file containing global simulation parameters. The fields relevant to LaRT are listed below.

Key	Type	Meaning
`ncpu`	integer	Number of MPI ranks used for this snapshot
`ndim`	integer	Number of spatial dimensions (always 3 for 3-D runs)
`nlevelmax`	integer	Maximum AMR refinement level used
`boxlen`	real	Box length in *code units* (see below)
`unit_l`	real	Length unit: 1 code length = `unit_l` cm
`unit_d`	real	Density unit: 1 code density = `unit_d` g cm^{-3}
`unit_t`	real	Time unit: 1 code time = `unit_t` s
`gamma`	real	Adiabatic index γ of the EOS

The velocity unit is derived as `unit_v` = `unit_l`/`unit_t` (cm s^{-1}).

Cosmological runs. For cosmological simulations, boxlen is in Mpc/h and unit_l incorporates the scale factor a so that L_{phys} = boxlen × unit_l cm is the comoving box size. LaRT reads the physical box size as L_{cm} = boxlen × unit_l.

Example info file (non-cosmological).

```
ncpu        =           64
ndim        =            3
nlevelmax   =           12
ngridmax    =       500000
boxlen      =   1.0000000000000000E+02
unit_l      =   3.0856775814913670E+21   ! 1 kpc in cm
unit_d      =   6.7707722919716770E-25
unit_t      =   3.0856775814913670E+14
gamma       =   1.6666666666666667E+00
```

In this example the box is 100 kpc on a side.

4. The AMR File (amr_NNNNN.outCCCCC)

Each CPU file stores the AMR tree data for that CPU's domain. The file consists of a *header section* followed by a sequence of *level blocks*, one per AMR level.

4.1 Header Records

The AMR header records are read in the following order:

Record	Variable(s)	Description
1	ncpu	Total number of MPI ranks
2	ndim	Number of dimensions
3	nx, ny, nz	Coarse grid size (typically $1 \times 1 \times 1$ or $2 \times 2 \times 2$)
4	nlevelmax	Maximum AMR level
5	ngridmax	Maximum number of grids (octs) per CPU
6	nboundary	Number of boundary domains
7	ngrid_current	Current number of active grids
8	boxlen	Box length (code units)
9–21	(various)	Cosmological parameters, time, etc. (skipped by LaRT)
22	ngridlevel(:,:)	Grid count array: (ncpu, nlevelmax)
23	(level sums)	Skipped
24–27	(boundary grids)	Only present if nboundary > 0 (skipped)
28–33	(coarse-grid / ordering arrays)	headf, ordering, Hilbert keys, coarse son, flag1, cpu_map; skipped by LaRT

The ngridlevel array. `ngridlevel(j, l)` gives the number of octs belonging to CPU j at AMR level l. LaRT constructs a combined table `ngridfile(j, l)` that includes both domain and boundary grids.

4.2 Level Blocks

After the header, the file contains one block per AMR level $l = 1, \ldots, \texttt{nlevelmax}$. Within each level, data are stored *domain by domain*: first all domains $j = 1, \ldots, \texttt{ncpu}$, then all boundary domains $j = \texttt{ncpu}+1, \ldots, \texttt{ncpu}+\texttt{nboundary}$. For each domain j at level l (if $\texttt{ngridfile}(j,l) > 0$):

Record(s)	Variable	Description
1	`ind_grid(:)`	Global grid indices
2	`next(:)`	Linked-list next pointer
3	`prev(:)`	Linked-list previous pointer
4–6	`xg(:, 1:3)`	Oct center coordinates in code units
7	`father(:)`	Parent grid index
8–13	`nbor(:,1:6)`	Neighbor grid indices (6 faces)
14–21	`son(:,1:8)`	Child grid indices; = 0 if a child is a **leaf**
22–29	`cpu_map(:,1:8)`	CPU ownership per child
30–37	`ref_map(:,1:8)`	Refinement flags per child

Record counts assume `ndim`= 3, `twotondim`= 8.

4.3 Oct Structure and Leaf Identification

RAMSES groups cells into *octs*: cubes of $2^3 = 8$ cells sharing a common parent grid cell. Each oct is identified by its *grid index*. Within an oct, the eight child cells are indexed $\texttt{ind} = 1, \ldots, 8$ with the following ordering:

$$i_x = (\texttt{ind}-1) \bmod 2, \quad i_y = \lfloor(\texttt{ind}-1)/2\rfloor \bmod 2, \quad i_z = \lfloor(\texttt{ind}-1)/4\rfloor, \tag{1}$$

so that `ind=1` is the $(-x, -y, -z)$ corner and `ind=8` is the $(+x, +y, +z)$ corner.

A child cell at index `ind` in grid g at level l is a **leaf** if and only if

$$\texttt{son}(g, \texttt{ind}) = 0. \tag{2}$$

Otherwise the value of `son` is the grid index of the finer oct that occupies that child cell.

4.4 Cell Position Calculation

The center of the coarse grid of the root domain is the origin. For a non-cosmological run with `nx=ny=nz=1`:

$$\mathbf{x}_{\rm bound} = \left(\frac{n_x}{2}, \frac{n_y}{2}, \frac{n_z}{2}\right). \tag{3}$$

The cell size at level l is $\Delta x_l = 2^{-l}$ (in units where $\texttt{boxlen} = 1$). The offset of child cell `ind` from the oct center is

$$\boldsymbol{\delta}_{\texttt{ind}} = \left(i_x - \tfrac{1}{2},\ i_y - \tfrac{1}{2},\ i_z - \tfrac{1}{2}\right) \times \Delta x_l. \tag{4}$$

The absolute position of child `ind` in grid g in fractional box units $[0,1]$ is

$$\tilde{x} = \frac{x_g + \delta_{\mathrm{ind},x} - x_{\mathrm{bound},x}}{n_x} + \frac{1}{2}, \tag{5}$$

and similarly for $\tilde{y}$ and $\tilde{z}$ (the $+1/2$ shifts the origin from the grid corner to the box corner). In the code:

```
xleaf = (xg(j,1) + xc(ind,1) - xbound(1)) / dble(nx) + 0.5d0
```

After reading, leaf positions are converted to physical units (cm):

$$x_{\mathrm{phys}} = \tilde{x} \times L_{\mathrm{box}}, \quad L_{\mathrm{box}} = \texttt{boxlen} \times \texttt{unit\_l}. \tag{6}$$

5. The Hydro File (`hydro_NNNNN.outCCCCC`)

The hydro file stores the conserved hydrodynamic variables for every cell (leaf and internal alike). It is organized in strict lockstep with the AMR file: one block per level per domain.

5.1 Header Records

Record	Variable	Description
1	`ncpu`	Number of MPI ranks
2	`nvar`	Number of hydro variables per cell
3	`ndim`	Number of dimensions
4	`nlevelmax`	Maximum AMR level
5	`nboundary`	Number of boundary domains
6	`gamma`	Adiabatic index

5.2 Variable Ordering

Standard RAMSES hydro stores the following variables in order:

Index	Variable	Physical meaning
1	ρ	Gas mass density [code units]
2	ρv_x	x-momentum density [code units]
3	ρv_y	y-momentum density [code units]
4	ρv_z	z-momentum density [code units]
5	E	Total energy density (kinetic + thermal) [code units]
6	$Z/Z_\odot$	Metallicity (if `metal=.true.`)
7+	passive scalars	Additional tracers (simulation-dependent)

RMHD / jellyfish variant. Some RAMSES derivatives provide a plain-text `hydro_file_descriptor.txt` file inside `output_NNNNN/`. The `jellyfish/RMHD` outputs inspected for this work use:

Index	Variable	Meaning
1	ρ	Gas density
2–4	v_x, v_y, v_z	*velocity* (not momentum)
5–10	$B_{\text{left/right}}$	Magnetic field components
11	P_{nt}	Non-thermal pressure
12	P_{th}	Thermal pressure
13+	passive scalars	Tracers / chemistry fields

In this case the velocity conversion omits the division by density and the temperature is reconstructed from thermal pressure rather than total energy.

Level block structure. For each level l and domain j, the hydro file contains:

1. Two header records (level number and domain number) — skipped.
2. For each cell index `ind`$= 1,\ldots,8$ and each variable `ivar`$= 1,\ldots,$`nvar`: one record of length `ngridfile`(j,l) containing the values for all grids of domain j at level l.

All records have the same element count `ngridfile`(j,l), so the loop structure is:

```
do ind = 1, twotondim      ! 8 cells per oct
  do ivar = 1, nvar        ! variables
    read(iu_hyd) var(:, ind, ivar)  ! (ngrida) values
  end do
end do
```

Precision. RAMSES writes hydro variables in double precision (`real*8`) by default. Some configurations compile with single precision (`real*4`). The LaRT reader handles both via the flag `ramses_hydro_prec` (set to 8 or 4 in `read_ramses_amr.f90`).

6. Physical Unit Conversions

6.1 Hydrogen Number Density

The mass density in code units is converted to CGS and then to hydrogen number density assuming a fully ionised gas with mean molecular weight $\mu_{\text{H}} \approx 1$ (one proton mass per hydrogen nucleus):

$$n_H\,[\text{cm}^{-3}] = \frac{\rho_{\text{code}} \times \texttt{unit\_d}}{m_H}, \quad m_H = 1.6726 \times 10^{-24}\ \text{g}. \tag{7}$$

The neutral fraction n_{HI}/n_H is computed separately in `grid_mod_amr.f90` using collisional ionisation equilibrium (CIE) if `par%use_cie_condition = .true.`, otherwise all hydrogen is assumed neutral.

6.2 Velocity

For standard RAMSES, variables 2–4 store momentum density ρv_α. The velocity in CGS is:

$$v_\alpha\ [\mathrm{cm\,s^{-1}}] = \frac{(\rho v_\alpha)_{\rm code}}{\rho_{\rm code}} \times \texttt{unit\_v}, \quad \texttt{unit\_v} = \frac{\texttt{unit\_l}}{\texttt{unit\_t}}. \tag{8}$$

For the RMHD variant described above, variables 2–4 are already *velocities* in code units, so LaRT instead uses

$$v_\alpha = v_{\alpha,\rm code} \times \texttt{unit\_v}. \tag{9}$$

LaRT receives velocities in km $\mathrm{s^{-1}}$ (divided by 10^5).

6.3 Temperature

For standard RAMSES, variable 5 is the *total energy density E* (kinetic plus thermal). The thermal energy density is:

$$e_{\rm th} = E - \frac{1}{2}\rho v^2 = E - \frac{(\rho v_x)^2 + (\rho v_y)^2 + (\rho v_z)^2}{2\rho}. \tag{10}$$

The specific thermal energy (per unit mass) in physical units is:

$$\varepsilon\ [\mathrm{erg\,g^{-1}}] = \frac{e_{\rm th}}{\rho_{\rm code}} \times \texttt{unit\_v}^2. \tag{11}$$

The temperature follows from the ideal gas law with adiabatic index γ and mean molecular weight $\bar{\mu}$ (LaRT assumes $\bar{\mu} = 1.22$ for neutral gas):

$$T\ [\mathrm{K}] = (\gamma - 1)\,\varepsilon\,\frac{\bar{\mu} m_H}{k_B}, \quad k_B = 1.381 \times 10^{-16}\ \mathrm{erg\,K^{-1}}. \tag{12}$$

A floor of $T = 10$ K is applied after conversion.

For the RMHD / jellyfish variant, the reader uses the thermal pressure `P_th` from `hydro_file_descriptor.txt` (variable 12 in the example outputs) and reconstructs the temperature directly from the ideal gas law:

$$T = \frac{P_{\rm th}}{\rho}\frac{\bar{\mu} m_H}{k_B}, \tag{13}$$

where

$$P_{\rm th}[\mathrm{erg\,cm^{-3}}] = P_{\rm th,code} \times \texttt{unit\_d} \times \texttt{unit\_v}^2. \tag{14}$$

7. The LaRT Reader: Step-by-Step

The reader in `read_ramses_amr.f90` proceeds as follows:

1. **Read `info` file.** Extract `ncpu`, `unit_l`, `unit_d`, `unit_t`, `boxlen`, and γ.
2. **Detect hydro layout.** If `output_NNNNN/hydro_file_descriptor.txt` exists, parse it to determine whether variables 2–4 are velocities or momenta and whether the thermodynamic variable is total energy or thermal pressure.
3. **Count leaf cells (first pass).** `ramses_count_leaves` opens every `amr_*.outCCCCC` file, reads the `son` arrays for all levels, and counts cells where `son` = 0. This gives the total $N_{\rm leaf}$ needed to pre-allocate output arrays.

4. **Read leaf data (second pass).** `ramses_read_all_cpus` opens *both* the AMR and hydro files simultaneously for each CPU:
   - Reads AMR grids level by level, extracting oct centers `xg` and the `son` array.
   - Reads hydro variables for the same grids.
   - For each child cell with `son` = 0 (leaf): computes its physical position, n_H, T, and $\mathbf{v}$, and appends them to the output arrays.
5. **Convert positions.** After the second pass, leaf positions (stored as fractions of `boxlen`) are multiplied by $L_{\rm cm}$ = `boxlen` × `unit_l` to give physical positions in cm. These are then passed to `amr_build_tree` as-is (with `distance2cm = 1`).
6. **Build octree and normalize.** Handled by `grid_mod_amr.f90` (see the AMR description document).

7.1 Domain Skipping

The AMR and hydro files interleave data from all $N_{\rm cpu}$ domains and boundary domains at each level. The LaRT reader only *stores* data for the domain that matches `icpu` (the loop variable); data from other domains are skipped with `skip_records`. This reproduces the original RAMSES read pattern used in post-processing tools such as PYMSES and YT.

8. File Naming Convention

File pattern	Content
`output_NNNNN/info_NNNNN.txt`	Global metadata
`output_NNNNN/amr_NNNNN.outCCCCC`	AMR tree for CPU `CCCCC`
`output_NNNNN/hydro_NNNNN.outCCCCC`	Hydro variables for CPU `CCCCC`
`output_NNNNN/gravity_NNNNN.outCCCCC`	Gravitational potential (optional)
`output_NNNNN/part_NNNNN.outCCCCC`	Particle data (optional)

where `NNNNN` is the zero-padded 5-digit snapshot index and `CCCCC` is the zero-padded 5-digit CPU rank (1-based).

9. LaRT Input for RAMSES Data

```
&parameters
 par%use_amr_grid  = .true.
 par%amr_type      = 'ramses'
 par%amr_file      = '/path/to/simulation'  ! repository directory
 par%amr_snapnum   = 10                     ! snapshot number (NNNNN)
 par%no_photons    = 1e6
 par%taumax        = 1.0e7
```

```
  par%DGR           = 0.0
  par%spectral_type = 'voigt'
  par%source_geometry = 'point'
  par%xs_point      = <x_cm>   ! source position in cm
  par%ys_point      = <y_cm>
  par%zs_point      = <z_cm>
  par%nxfreq  = 121
  par%nxim    = 64
  par%nyim    = 64
/
```

Notes.

- `par%amr_file` is the *repository directory* containing `output_NNNNN/` (not the output directory itself).
- `par%amr_snapnum` selects the snapshot (the integer `NNNNN`).
- For RAMSES data, `par%distance2cm` is automatically set to 1.0 because the reader returns positions in cm (pre-multiplied by `unit_l`). The `par%distance_unit` parameter is ignored.
- `par%taumax` is specified as the line-center optical depth from the box center to the $+z$ boundary.
- Source position (`xs_point` etc.) must be given in cm, matching the units of the RAMSES leaf positions after conversion.
- If `par%use_cie_condition = .true.`, the neutral fraction $n_{\rm HI}/n_H$ is computed from the temperature via collisional ionisation equilibrium; otherwise all hydrogen is assumed neutral.

10. Generic Text Format vs. RAMSES Binary

For cases where RAMSES binary output is unavailable (e.g., data from other simulation codes, or hand-constructed test grids), LaRT provides the generic text format described separately. The two formats differ in the following ways:

Property	RAMSES binary	Generic text
File format	Fortran unformatted binary	Plain text
Position unit	cm (auto, from `unit_l`)	Code units (set by `distance_unit`)
`par%amr_type`	`'ramses'`	`'generic'`
`par%amr_snapnum`	required	not used
Multi-CPU support	yes (reads all `.outCCCCC` files)	no (single file)
Physical units	full CGS conversion built in	user's responsibility

The generic text file format is:

```
nleaf  boxlen
x  y  z  level  nH[cm^-3]  T[K]  vx[km/s]  vy[km/s]  vz[km/s]
...
```

The Python utility `examples/python/make_amr_grid.py` provides the `AMRGrid` class to construct and write these files from arbitrary density, temperature, and velocity fields.

11. Known Limitations and Assumptions

1. **Neutral fraction.** The RAMSES reader sets n_H (total hydrogen density, not $n_{\rm HI}$). The neutral fraction is computed later in `grid_create_amr` using either CIE (if `par%use_cie_condition = .true.`) or the assumption of full neutrality. For simulations with radiation feedback or on-the-fly ionisation, a custom neutral fraction field from a passive scalar may be needed.

2. **Mean molecular weight.** The temperature calculation in `ramses_read_all_cpus` assumes $\bar{\mu} = 1.22$ (fully neutral primordial gas). For ionised or metal-enriched gas this gives an overestimate.

3. **Hydro precision.** The variable `ramses_hydro_prec` (default 8) must be changed to 4 for simulations compiled with single-precision hydro.

4. **Non-standard variable ordering.** If the RAMSES simulation includes passive scalars before the thermodynamic variable, the standard assumption (E = var 5) may be wrong. The current reader first checks `hydro_file_descriptor.txt`; if that file is absent, standard RAMSES ordering is assumed.

5. **Memory.** All leaf cells are broadcast to every MPI rank in LaRT. For large RAMSES outputs ($\gtrsim 10^7$ leaf cells) this may require significant RAM per node.

References

[1] Teyssier, R. 2002, A&A, 385, 337 (*RAMSES: a new high-resolution n-body and hydrodynamics code for cosmological simulations*)